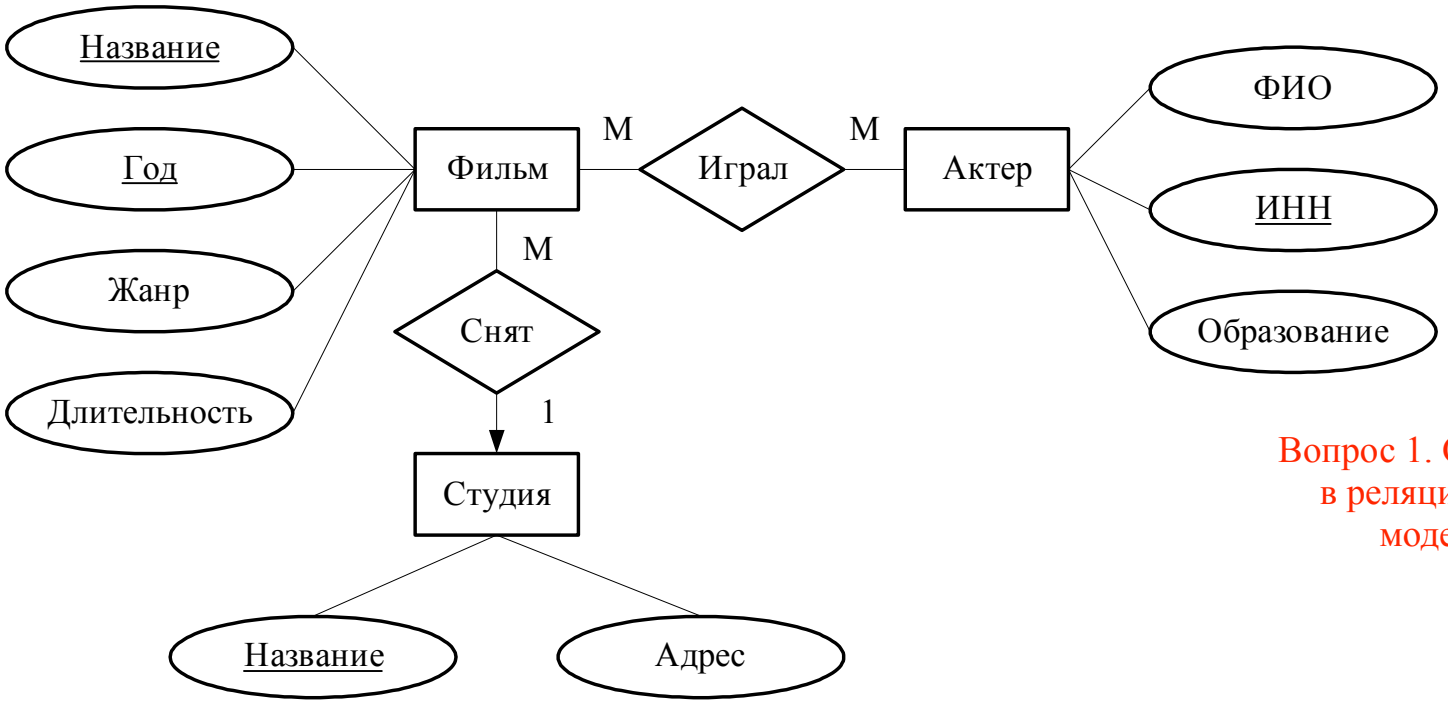
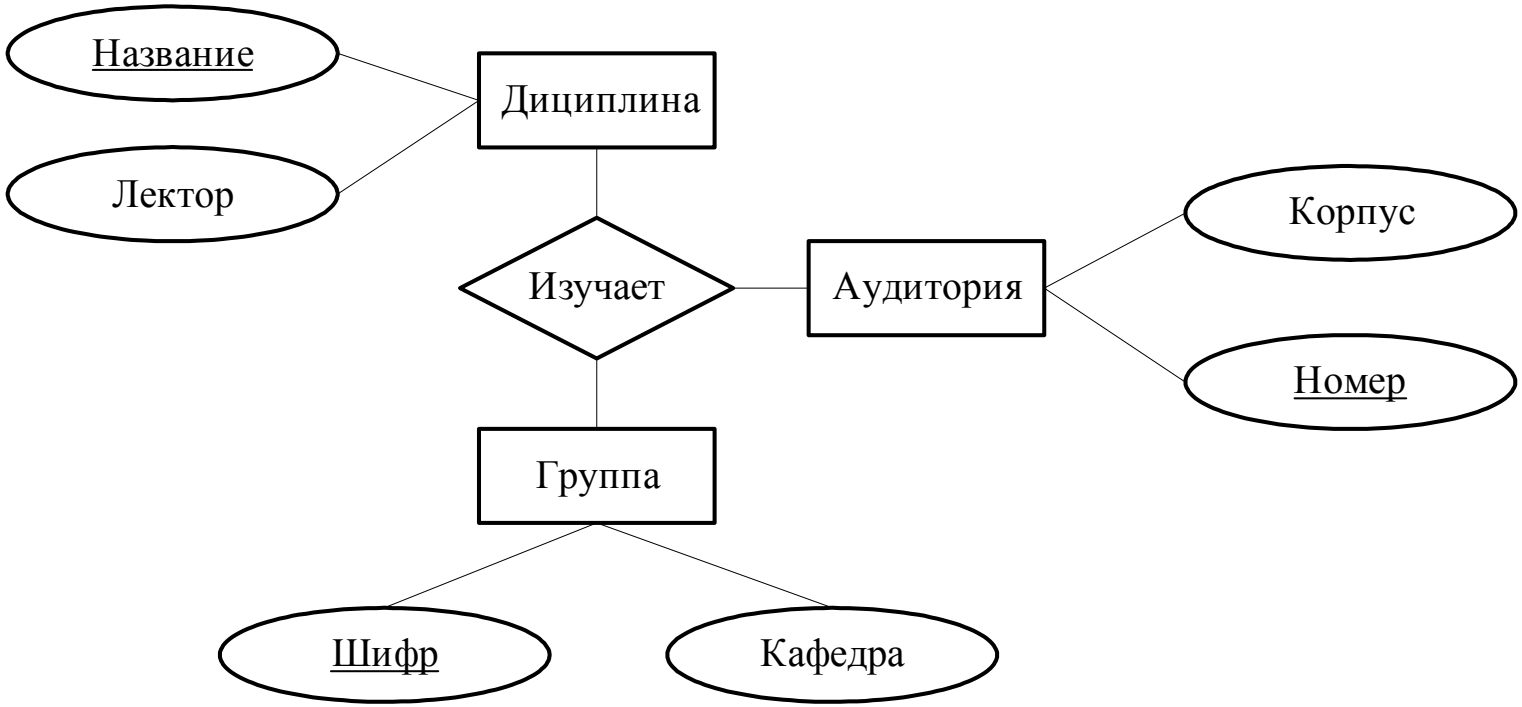


Вопрос 1. Схема БД в реляционной модели

- Актёр (ИНН, Ф.И.О, Образование)
- Фильм (Название, Год, Длительность, Жанр, Студия)
- Студия (Название, Адрес)
- Актёр–Фильм (Название, Год, ИНН)



- Дисциплина (Название, Лектор)
- Группа (Шифр, Кафедра)
- Аудитория (Номер, Корпус)
- Занятие (Дисциплина, Группа, Аудитория)

Ограничения

- Для столбца:
  - **NOT NULL** – значение всегда не пустое ;
  - **UNIQUE** — все значения столбца должны быть уникальны;
  - **PRIMARY KEY** — первичный ключ и уникальность;
  - **FOREIGN KEY** — внешний ключ,
  - **CHECK** (условие) —условие для проверки значения столбца. В скобках м.б. условие, выборка, запрос.
- Ограничения для таблицы:
  - **CHECK** — ограничение на значения полей кортежа;
  - **PRIMARY KEY** —первичный ключ,
  - **FOREIGN KEY** — ограничение ссылочной целостности.

2. Объектно-реляционная модель (UDT)

sql

Copy Download

```sql
-- UDT типы
CREATE TYPE AddressType AS (
    city VARCHAR(50),
    street VARCHAR(50)
);

CREATE TYPE UserType AS (
    user_id INTEGER,
    name VARCHAR(30),
    addr AddressType,
    reg_date DATE
);

CREATE TYPE AdType AS (
    ad_id INTEGER,
    header VARCHAR(100),
    category VARCHAR(30),
    price DECIMAL(10,2)
);

-- Типизированные таблицы
CREATE TABLE Users OF UserType (
    user_id WITH OPTIONS PRIMARY KEY
);

CREATE TABLE Ads OF AdType (
    ad_id WITH OPTIONS PRIMARY KEY,
    user_id_ref REF(UserType) SCOPE Users
);

CREATE TABLE Recommendations (
    rec_id INTEGER PRIMARY KEY,
    ad_ref REF(AdType) SCOPE Ads,
    user_ref REF(UserType) SCOPE Users,
    rec_date DATE,
    relevance_score FLOAT
);
```

Просто ответ дипсика
(не является эталоном решения)

Орг. целостности: REF SCOPE, PRIMARY KEY, NOT NULL.

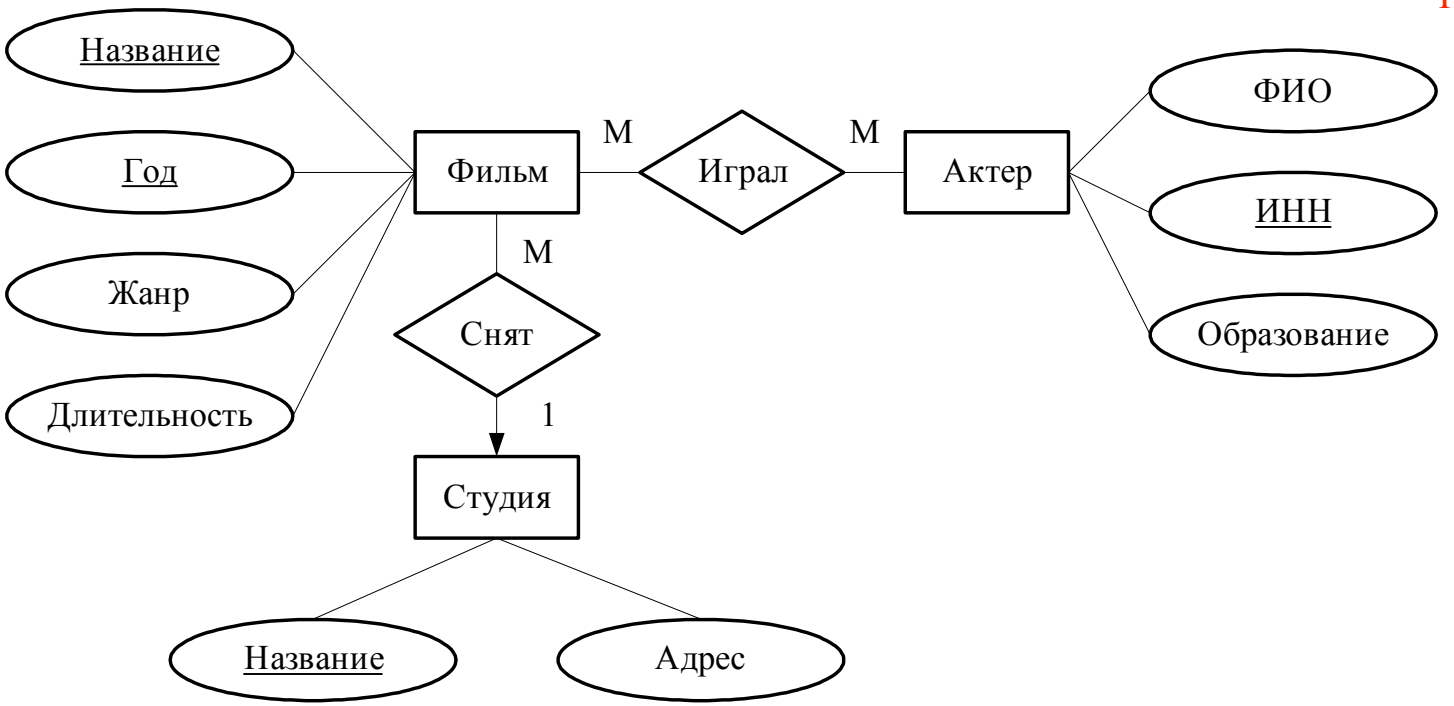
Ссылки: user_id_ref, ad_ref, user_ref.

Пример типизированной таблицы

```
CREATE TYPE ActorT AS
(
    inn char(10),
    fio ROW ( f varchar(10), i varchar(10), o varchar(10)),
    edu varchar(15)
);

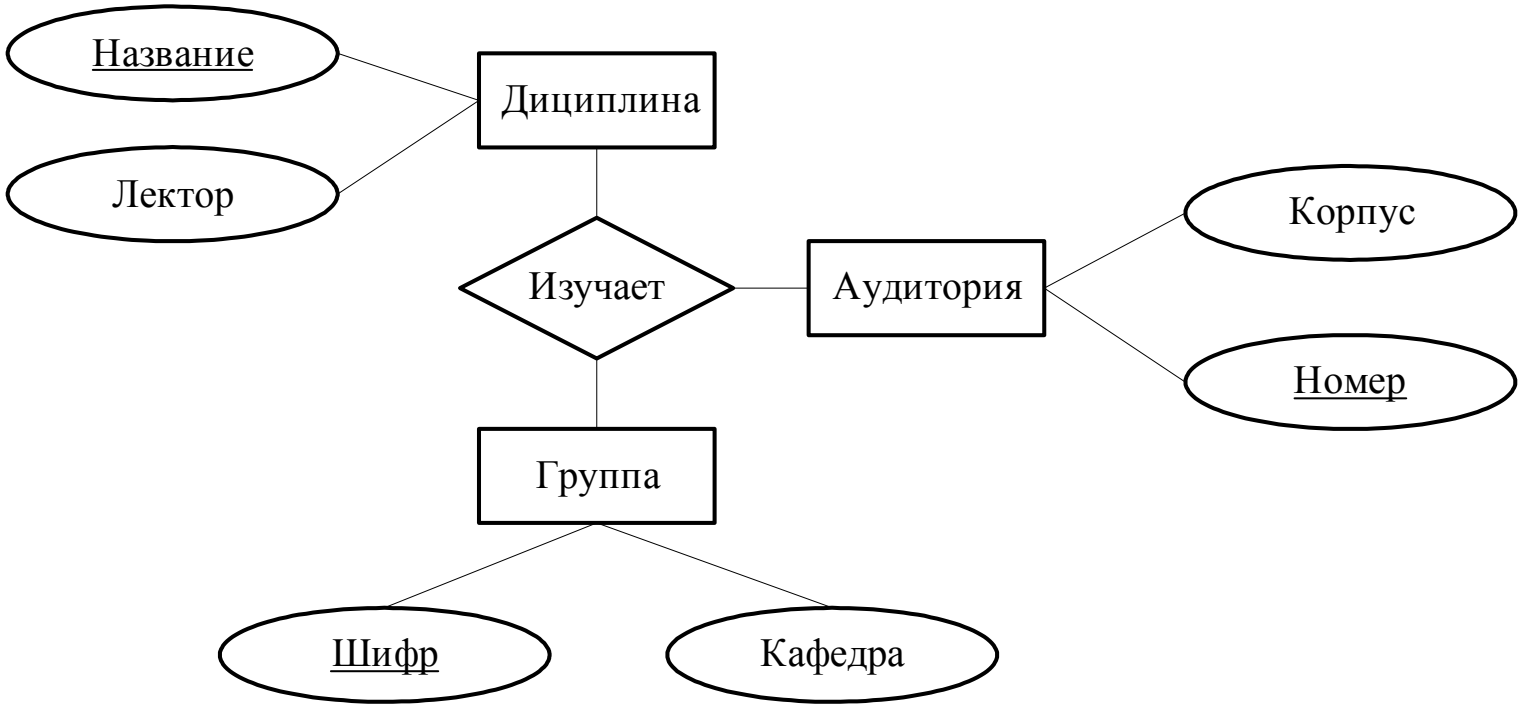
CREATE TABLE Actor OF ActorT
(
    PRIMARY KEY (inn),
    REF IS Ida system generated
);
```

Пример 1



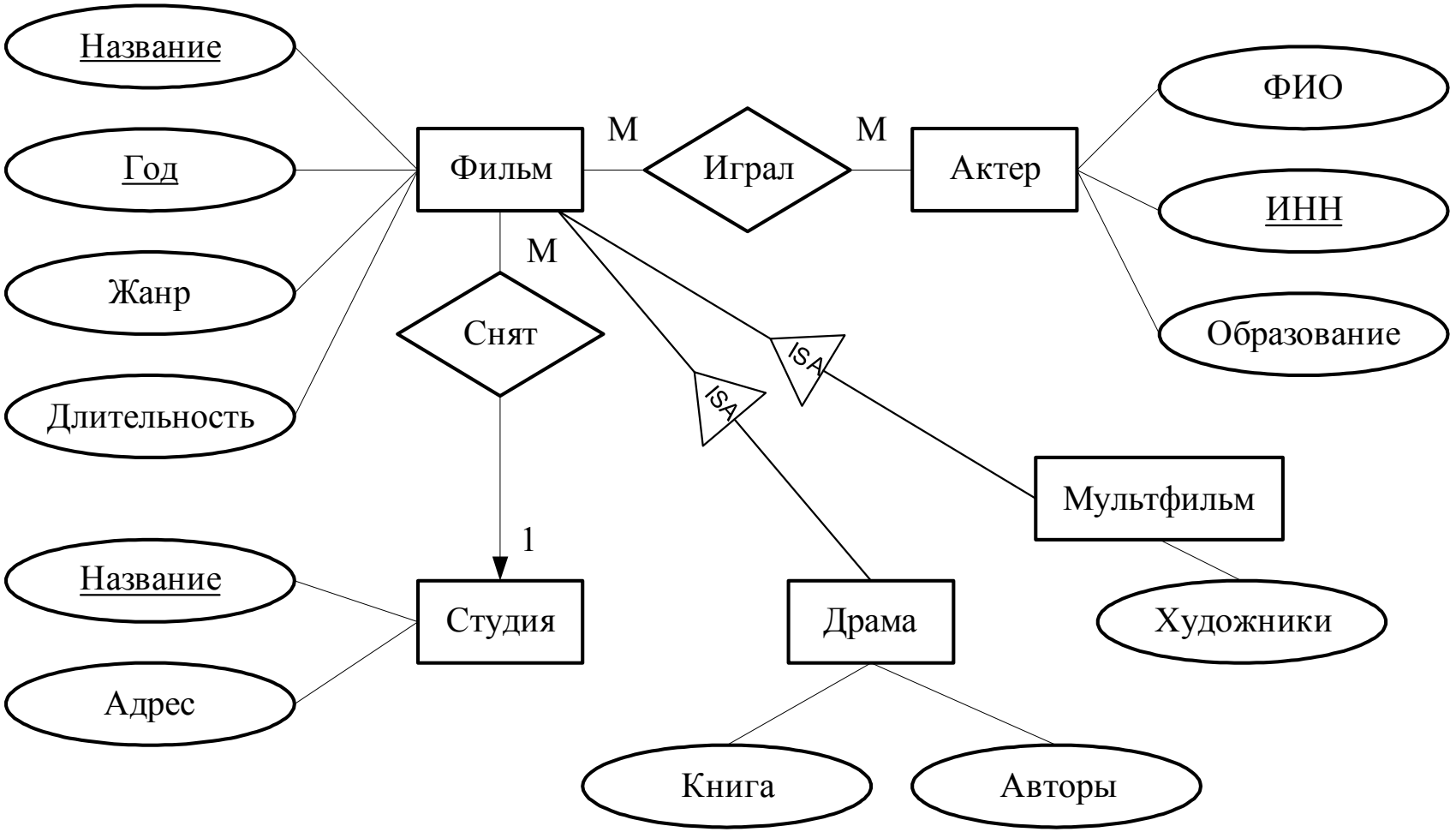
- Актёры (ИНН, Ф.И.О (Фамилия, Имя, Отчество), Образование)
- Фильмы (Название, год, Длительность, Жанр, Студия (*Студии))
- Студии (Название, Адрес (Город, Страна))
- Актёр–Фильм (Фильм (*Фильмы), Актер(*Актеры))

Пример 2



- Дисциплина (Название, Лектор)
- Группа (Шифр, Кафедра)
- Аудитория (Номер, Корпус)
- Занятие (Дисц (*Дисциплина), Группа(*Группа), Ауд (*Аудитория)

Пример 3


Название
Год
Жанр
Длительность
Фильм
M
Играл
M
Актер
ФИО
ИНН
Образование
M
Снят
1
ISA
ISA
Мультфильм
Художники
Название
Адрес
Студия
Драма
Книга
Авторы

Актёры (ИНН, Ф.И.О (Фамилия, Имя, Отчество), Образование)

Фильмы (Название, год, Длительность, Жанр, Студия (*Студии))

Студии (Название, Адрес (Город, Улица))

Актёр–Фильм (Фильм (*Фильмы), Актер (*Актеры))

Мультфильмы: Фильмы (Название, Год, Длительность, Жанр, Студия (*Студия), Художники)

Драмы: Фильмы (Название, Год, Длительность, Жанр, Студия (*Студия), Книга, Авторы)

Переход по ссылке (1-M)

```
SELECT year, length,
        stud–>sname,
        stud–>addr.city,
        stud–>addr.fullad()
FROM TF
WHERE title = 'The Matrix'
```

Переход по ссылке (M-M)

```
SELECT film–>title,
                film–>year,
                film–>stud–>name
FROM FA
WHERE act–>fio='Иванов'
```

Разыменование ссылки

```
SELECT deref( act )
FROM FA
WHERE film–>title='The Matrix'
```

3. Объектная модель (ODL)

odl (extent Users) Copy Download

```
class User {
    attribute integer user_id;
    attribute string name;
    attribute struct { string city; string street; } address;
    attribute date reg_date;
    relationship set<Ad> ads inverse Ad::owner;
};

class Ad {
    attribute integer ad_id;
    attribute string header;
    attribute string category;
    attribute float price;
    relationship User owner inverse User::ads;
    relationship set<Recommendation> recs inverse Recommendation::ad;
};

class Recommendation {
    attribute integer rec_id;
    attribute date rec_date;
    attribute float relevance;
    relationship User recommended_user inverse User::???;
    relationship Ad ad inverse Ad::recs;
};
```

Экстенты: extent Users, extent Ads, extent Recommendations.

Экстент класса

- **Экстент** — множество объектов данного типа (класса) в базе данных.

- Если класс A является производным классом от базового класса B, экстент A — подмножество экстента B.
- Объект производного класса также является объектом базового класса.
- Если объект – аналог кортежа, экстент – аналог таблицы

Описание классов на языке ODL

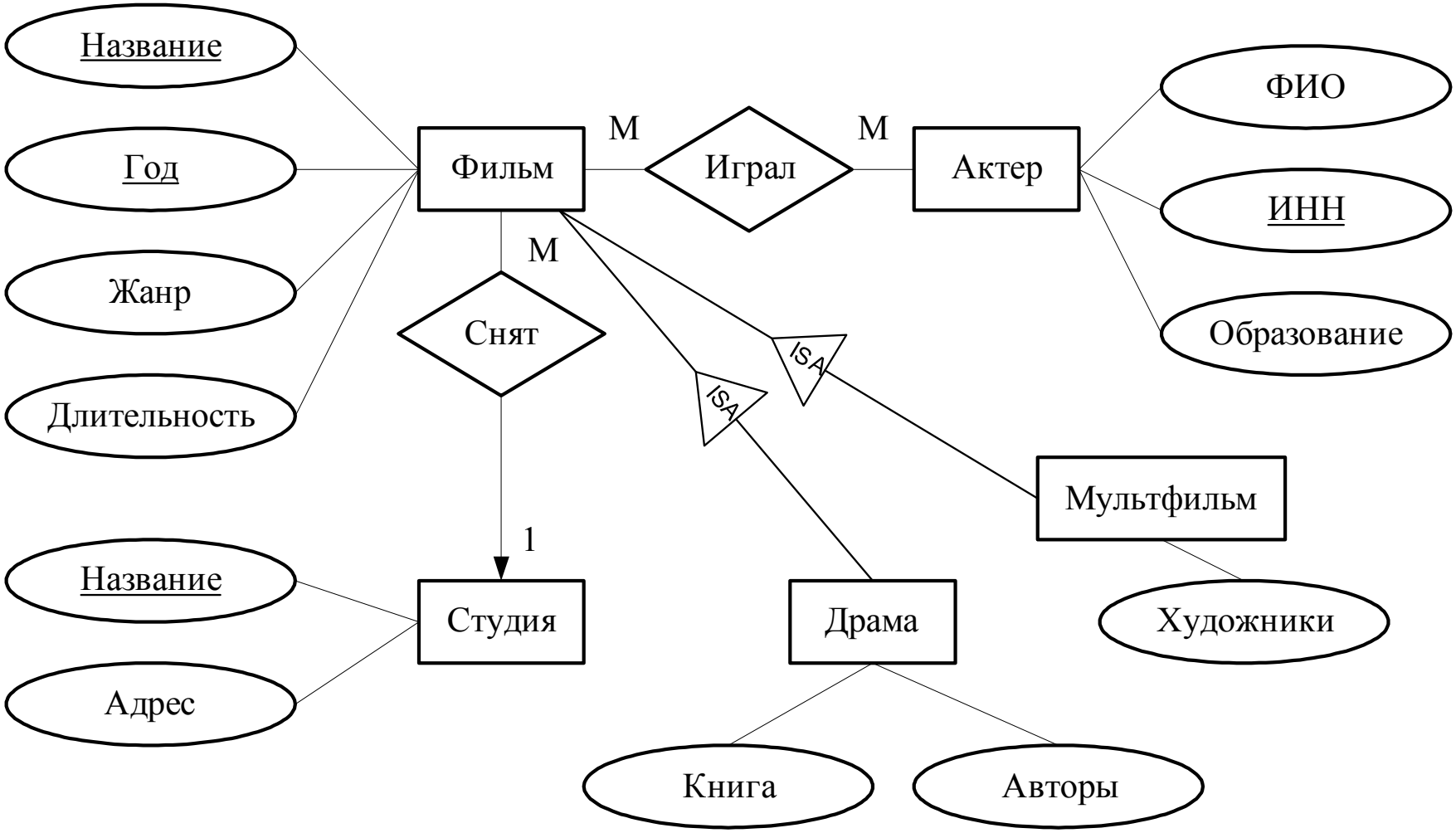
- Язык ODL используют для описания интерфейса базы данных. Реализация базы данных выполняется на языке OML.
- Описание объектного типа (класса) состоит их следующих элементов:
  - **имя** (название) класса;
  - набор **супертипов** (базовых классов или интерфейсов);
  - имя поддерживаемого системой **экстента** (extent);
  - один или более **ключей** (key или keys);
  - набор **атрибутов**, каждый из которых может быть объектом или литеральным значением;
  - набор **связей**, каждая из которых указывает на некоторый другой объект или множество объектов;
  - набор **методов**, определяемых своими сигнатурами.

Конструкция для описания класса на языке ODL

class название
(**extent** название_экстента **key** ключи)
{
атрибуты;
методы;
связи;
};

Для возможности создания объектов класса обязательно указывают его экстент.

Пример ER-модели



Описание класса Фильм

```
class Film(extent Films key(name, year))
{
     attribute string name;
     attribute integer year;
     attribute integer len;
     attribute enum Ftype {bw, color} type;

    relationship Studia stud
    inverse Studia::fs;
    relationship Set<Actor> acts
    inverse Actor::infs;

    integer ActCount() raises(noActors);
    void FIlmsInYear(in integer year, out Set<Films>)
    raises(noFilms, badYear);
};
```

Описание класса Студия

```
class Studia (extent Studies)
{
 attribute string sname;
 attribute Struct Addr
    {
     string city,
     string street
    } addr;

    relationship Set<Film> fs
                inverse Film::stud;
};
```

Описание класса Актер

```
class Actor(extent Actors  key inn)
{
 attribute string inn;
 attribute string fio;
 attribute List<string> edu;

    relationship Set<Film> infs
            inverse Film::acts;
};
```

Описание производных классов

```
class MF extends Film (extent MFS)
{
    attribute string drawer;
};

class Drama extends Film (extent Dramas)
{
    attribute Struct {
            Set<string> authors,
            string bname }   book;
};
```

Переход по связи от М к 1

- названия фильмов и выпустивших их студий, расположенных в Москве

```
SELECT f.name,
                f.stud.sname
FROM Films f
WHERE
        f.stud.addr.city = “Москва”
```

Переход по связи от 1 к М

- названия фильмов и выпустивших их студий, расположенных в Москве (запрос можно рассматривать как выполнение вложенных циклов).

```
SELECT f.name,
                f.stud.sname
FROM Studies s,
          s.fs   f
WHERE
          s.addr.city = “Москва”
```

Переход по связи M-M

- Ф.И.О актеров, игравших в фильме остров

```
SELECT    a.fio
FROM      Actors a,  a.infs  r
WHERE
          r.name = “Остров”
```

Правильный DTD (соответствующий приведённому XML)

XML-документ:

xml Copy Download | Run

```xml
<recommendations_system>
  <user>
    <user_id>1</user_id>
    <name>Анна</name>
    <address>
      <city>Москва</city>
      <street>Тверская</street>
    </address>
    <reg_date>2024-01-10</reg_date>
  </user>
  <ad>
    <ad_id>101</ad_id>
    <header>Ноутбук</header>
    <category>электроника</category>
    <price>50000</price>
    <user_id_ref>1</user_id_ref>
  </ad>
  <recommendation>
    <rec_id>1001</rec_id>
    <ad_id_ref>101</ad_id_ref>
    <user_id_ref>2</user_id_ref>
    <rec_date>2024-02-01</rec_date>
    <relevance>0.95</relevance>
  </recommendation>
</recommendations_system>
```

Задание №4
Просто пример дипсика - не является эталоном

DTD для этого XML:

Язык DTD

- DTD (Document Type Definition) — определение типа документа для автоматизированной обработки XML-документа.
- Документ в формате DTD задает набор допустимых тегов, их атрибутов и правил вложения

Описание элементов

- Каждый элемент XML должен быть описан в DTD конструкцией :

  <!ELEMENT название содержимое>
- Содержимое элемента указывают из следующего списка:

  (#PCDATA) — текст, нет вложенных элементов;

  (el1, el2, el3) — содержит последовательность элементов el1, el2 и el3;

  EMPTY — пустой элемент;

  ANY — любое содержимое.
- Сложная структура вложенных элементов

  * ноль или более раз;

  + один или более раз;

  ? ноль или один раз;

  | один из указанных элементов.
- Пример элемента E, содержит либо текст, либо набор street и city:

  <!ELEMENT E (# PCDATA | (street*,city*)) >

Запись в XML связей

- Связь между объектами задают через атрибуты, идентификаторы и ссылки на идентификаторы.
- Если n и m — объекты (вершины) на графе и имеется связь между ними от n к m, то в XML помещают элементы:

<m mid=”m1”>....</m>

<n tom=”m1”>....</n>

- где mid – атрибут типа ID (идентификатор объекта m);
- tom – атрибут типа IDREF (ссылка на идентификатор объекта m).

Описание связей в DTD

- В DTD-определении записывают:

<!ELEMENT n …. >

<!ELEMENT m …..>

<!ATTLIST m mid ID>

<!ATTLIST n tom IDREF>

Пример XML

```
<?xml version="1.0" encoding="UTF8" standalone="no"?>
<!DOCTYPE db  SYSTEM  "file.dtd">
<db>
    <film idf="f1" toact="a1 a2" tos="s1">
                <name>Чапаев</name>
                <year>1934</year>
                <len>63</len>
                <type>драма</type>
    </film>
    <actor ida="a1" tof="f1">
                <fio>Иванов И.И. </fio>
                <edu>Щукинское училище</edu>
    </actor>
    <actor ida="a2" tof="f1">
                <fio>Бабочкин Б.А.</fio>
                <edu>Студия Певцова</edu>
    </actor>
    <studio ids="s1" tof="f1">
                <name>Ленфильм</name>
                <addr>Санкт-Питербург, Россия</addr>
    </studio>
</db>
```

Пример DTD

```
<!ELEMENT db (film*, actor*, studio+)>
<!ELEMENT film (name, year, len?, type?)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT len (#PCDATA)>
<!ELEMENT type (#PCDATA)>
<!ELEMENT actor (fio, edu+)>
<!ELEMENT fio (#PCDATA)>
<!ELEMENT edu (#PCDATA)>
<!ELEMENT studio (name, addr )>
<!ELEMENT addr (#PCDATA)>
<!ATTLIST film idf ID #REQUIRED toact IDREFS tos IDREF>
<!ATTLIST actor ida ID tof IDREFS>
<!ATTLIST studio ids ID tof IDREFS>
```

Язык XPath

- Языки XPath и XQuery используют для составления запросов к данным в XML.
- Оба языка разработаны консорциумом W3C.
- Язык XPath (XML Path Language) — язык запросов к элементам XML документа.
- Предназначен для навигации по XML-документу, позволяет указать путь к элементу, условие выборки и извлекаемое значение.

Сокращения в путевом выражении

- descendant (потомки) — «//»;
- parent (родитель) — «..»;
- self (сам элемент) — «.»;
- child (дети) — по умолчанию без обозначения;
- attribute — атрибут элемента «@имя».

Язык XQuery

- Язык XQuery — язык запросов к XML-документам, включает в себя язык XPath 2.0.
- Кроме навигации по XML-документу позволяет генерировать новые элементы, определять и выполнять пользовательские функции.
- Содержит конструкции для определения циклов, условий, сортировки, работы с переменными.

Описание связей в DTD

- В DTD-определении записывают:

<!ELEMENT n …. >

<!ELEMENT m …..>

<!ATTLIST m mid ID>

<!ATTLIST n tom IDREF>